Programming II

Libraries and Java Strings

Michele Pasqua, PhD

UNIVERSITÀ
di VERONA
Dipartimento
di INFORMATICA

A.Y. 2025/2026

Libraries in Java

Standard libraries

Java provides a large amount of standard libraries (or packages)

- `java.lang` // *core runtime features*
- `java.io` // *input/output support*
- `java.math` // *mathematical operations*
- `java.awt` // *basic GUI support*
- `java.util` // *utility functions*
- ...

For instance, `java.lang.String` provides support for strings

Library import

Before use, libraries or classes must be imported

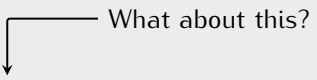
```
import java.util.*;   or   import java.util.Scanner;
```

Note that

- The library `java.lang` is always imported by default
- Libraries and classes must be available to the classpath

Standard library import

```
import java.util.Scanner; // import a standard library

public class MainScanner {

  public static void main(String[] args) {
    Scanner myScanner = new Scanner(System.in);

    System.out.println("Enter username: ");
    String userName = myScanner.nextLine();

    System.out.println("The username you typed is: " + userName);
    myScanner.close();
  }

}
```



Compile with `javac -cp . MainScanner.java`

User-defined class import

For now, put all `.class` files under the same classpath

- We will see later how to package and import user-defined libraries

```
// no need import directives if classes are in the same classpath

public class MainImport {

  public static void main(String[] args) {
    System.out.println("We␣will␣import␣MainScanner␣and␣run␣its␣main");
    MainScanner.main(); // the main is a static method
  }

}
```

Compile with `javac -cp . MainImport.java`

Some standard Java classes

The `java.lang.System` class provides the constants

- `err`, which refers to the standard error
- `in`, which refers to the standard input
- `out`, which refers to the standard output

The `java.lang.System` class provides the static method `currentTimeMillis()`

- It returns a `long`
- It returns the milliseconds elapsed from January 1, 1970 (UTC) at midnight

Unix time


WE DID IT, WE TIME TRAVELED!
BUT TO WHAT YEAR?
LET ME ASK THAT GUY OVER THERE
HEY!
What's Today's date?
January 1 1970
We are at the begining of history.
SRGRAFO

Some standard Java classes (cont'd)

To use the `java.util.Scanner` class

- `sc = new Scanner(source)`, a scanner object must be attached to a source
- `sc.close()`, a scanner object should be closed after use

The `java.util.Scanner` class provides the methods

- `nextLine()`, reading a line from the source as a `String`
- `nextInt()`, reading an integer from the source as a `int`
- `nextFloat()`, reading a decimal from the source as a `float`
- ...an exception will be thrown in case of type mismatch

Strings in Java

String class

In Java, strings are instances of the `String` class

We can create strings of two kinds

- Empty strings, with `String str = new String();`
- Non-empty strings, with `String str = new String("a␣string");`

Here `"a␣string"` is a constant string

- The compiler implicitly transforms it into a `String` object
- One can also write `String str = "";` and `String str = "a␣string";`

String class (cont'd)

In Java, strings are not (null-terminated) arrays of characters

```
char carr[] = {'h', 'e', 'l', 'l', 'o'};  ≠  String str = "hello";
```

All strings are `String` objects

- We can have `String str = null;`
- We cannot modify strings, `String` objects are immutable
- But we can copy strings

```
String s = new String("abc");                                    1
String t = new String(s);                                        2
// s and t refer to two different objects, representing the same string   3
```

String concatenation

The only operator available on strings is + performing concatenation

```
String s = new String("Hello␣");
String t = s + "World";
System.out.println(t + "!")
// the output will be the 'Hello World!'
```

All other operations on strings are implemented as method of the `String` class

Operations on strings

The number of elements in a string is computed by

`str.length()`

```
String str = "";
System.out.println(str.length()); // the output will be the '0'

String str = "hello";
System.out.println(str.length()); // the output will be the '5'

System.out.println("abc".length()); // the output will be the '3'
```

Operations on strings (cont'd)

One can access the i-th element of a string by

```
str.charAt(i)
```

Indexing starts from 0, if the index is out-of-bounds an exception is thrown

```
String str = "hello";

System.out.println(str.charAt(0)); // the output will be the 'h'

System.out.println(str.charAt(4)); // the output will be the 'o'

System.out.println(str.charAt(5)); // the JVM will throw an exception
```

Operations on strings (cont'd)

One can access the `i`-th element of a string by

```
str.charAt(i)
```

`String` objects are immutable, it is not possible to modify string elements

```
String str = "hello";

System.out.println(str.charAt(3)); // the output will be the 'l'

str.charAt(3) = 'm'; // compilation error
```

String comparison

Being objects, the == operator is applied to references when strings are compared

`s == t` is *true* only if `s` and `t` point to the same object

```
String s = "hello";
String t = new String("hello");
String r = s;

System.out.println(s == t); // the output will be 'false'

System.out.println(s == r); // the output will be 'true'
```

String comparison (cont'd)

To compare strings, use the method `equals` provide by `String` class

`s.equals(t)` is *true* when `s` and `t` represent the same string

```
String s = "hello";
String t = new String("hello");
String r = s;

System.out.println(s.equals(t)); // the output will be 'true'

System.out.println(s.equals(r)); // the output will be 'true'
```

String comparison (cont'd)

Pay attention to compiler optimizations

- Always use `equals` to compare strings

```
String s = "hello";
String t = "hello";
String r = s;

System.out.println(s == r); // the output will be 'true', expected

System.out.println(s == t); // the output will be 'true', why?
```

More operations on strings

The index (first occurrence) of a character `c` in a string is computed by

`str.indexOf(c)`

The method returns -1 when the character is not present

```
String str = "hello";

System.out.println(str.indexOf('e')); // the output will be the '1'

System.out.println(str.indexOf('j')); // the output will be the '-1'
```

More operations on strings (cont'd)

The substring between the bounds `start` and `end` in a string is computed by

`str.substring(start, end)`

The method returns the string starting at index `start` and ending at index `end-1`

```
String str = "hello";

System.out.println(str.substring(1, 3)); // the output will be the 'el'
```

More operations on strings (cont'd)

The substring between the bounds `start` and `end` in a string is computed by

```
str.substring(start, end)
```

If `end` is omitted, the substring goes until the of the string

```
String str = "hello";

System.out.println(str.substring(1, 3)); // the output will be the 'el'

System.out.println(str.indexOf(2)); // the output will be the 'llo'
```

More operations on strings (cont'd)

The substring between the bounds `start` and `end` in a string is computed by

```
str.substring(start, end)
```

If `start` or `end` are out-of-bounds an exception is thrown

```
String str = "hello";

System.out.println(str.substring(1, 3)); // the output will be the 'el'

System.out.println(str.indexOf(2)); // the output will be the 'llo'

System.out.println(str.indexOf(2, 6)); // the JVM will throw an exception
```

More operations on strings (cont'd)

Replace the string `target` with the string `replacement` in a string `str` with

```
str.replace(target, replacement)
```

Replacement is applied from left to right (possibly multiple times)

```
String str = "hello";

System.out.println(str.replace("lo", "ix")); // outputs 'helix'

System.out.println(str.replace("o", ""));    // outputs 'hell'

System.out.println(str.replace("l", "ll"));  // outputs 'hellllo'
```

Implicit cast

In Java, one can concatenate strings and integers (or other primitive types)

- The compiler performs an implicit cast from `int` to `String`
- When using +, the compilers automatically deals with `null` values

```
String str = "Result␣=␣" + 3;

System.out.println(str); // the output will be 'Result = 3'

System.out.println(str + null); // the output will be 'Result = null'
```

Actually, the compiler calls the `valueOf()` method of the `String` class

Strings à la C in Java

The `String` class provides the method `format()`

- The syntax is similar to the format strings of C *// used in printf et simila*

```
String str = "Result";                                                    1
int num = 3;                                                              2
                                                                          3
String formatStr = String.format("%s = %d", str, num);                    4
                                                                          5
System.out.println(formatStr); // the output will be 'Result = 3'         6
```

Mutable strings

The `java.lang` package provides the class `StringBuilder`

- Create objects that hold a mutable sequence of characters

Immutable `String`

```
String str = "abc";                                                    1
str = str + "def"; // no actual append, a new object is created        2
```

Mutable `StringBuilder`

```
StringBuilder sb = new StringBuilder("abc");                           1
sb.append("def"); // append on the same object                         2
```

Mutable strings (cont'd)

Useful to dynamically craft strings

```
String printArray(int[] arr) { // if arr is '[1, 5, 6]'
  StringBuilder sb = new StringBuilder();

  sb.append("Array(").append(arr.length).append(")[");
  for (int i = 0; i < arr.length; i++) {
    sb.append("␣").append(arr[i]);
  }
  sb.append("␣]");

  return sb.toString(); // the method will return 'Array(3)[ 1 5 6 ]'
}
```

Mutable strings (cont'd)

Some additional operations // *all in fluent notation*

Delete a substring within bounds (right-exclusive) with

```
StringBuilder delete(int start, int end);
```

Insert a string at position with

```
StringBuilder insert(int index, String str);
```

Reverse a string with `StringBuilder reverse();`

Q + A